

```
In [1]: 1 class Sorting:
2       def bubble(self,arr):
3           l=len(arr)
4           for i in range(l):
5               flag=1
6               for j in range(l-i-1):
7                   if arr[j+1]<arr[j]:
8                       arr[j+1],arr[j]=arr[j],arr[j+1]
9                   flag=0
10              if flag==1:
11                  return arr
12              return arr
13 obj=Sorting()
14 obj.bubble([5,2,7,9,1,0,3,4,8,6,6,3])
```

Out[1]: [0, 1, 2, 3, 3, 4, 5, 6, 6, 7, 8, 9]

```
In [3]: 1 class Sorting:
2       def insertion(self,arr):
3           l=len(arr)
4           for i in range(1,l):
5               val=arr[i]
6               gap=i
7               while gap>0 and arr[gap-1]>val:
8                   arr[gap]=arr[gap-1]
9                   gap-=1
10              arr[gap]=val
11              return arr
12 obj=Sorting()
13 obj.insertion([5,2,7,9,1,0,3,4,8,6,6,3])
```

Out[3]: [0, 1, 2, 3, 3, 4, 5, 6, 6, 7, 8, 9]

```
In [6]: 1 class Sorting:
2       def selection(self,arr):
3           l=len(arr)
4           for i in range(l):
5               ind=i
6               for j in range(i+1,l):
7                   if arr[j]<arr[ind]:
8                       ind=j
9               arr[ind],arr[i]=arr[i],arr[ind]
10              return arr
11 obj=Sorting()
12 obj.selection([5,2,7,9,1,0,3,4,8,6,6,3])
```

Out[6]: [0, 1, 2, 3, 3, 4, 5, 6, 6, 7, 8, 9]

```
In [3]: 1 class Sorting:
2       def quick(self,arr):
3           l=len(arr)
4           if l<=1:
5               return arr
6           else:
7               pivot=arr[0]
8               left=[i for i in arr[1:] if i<pivot]
9               right=[i for i in arr[1:] if i>=pivot]
10              return self.quick(left)+[pivot]+self.quick(right)
11 obj=Sorting()
12 obj.quick([5,2,7,9,1,0,3,4,8,6,6,3])
```

Out[3]: [0, 1, 2, 3, 3, 4, 5, 6, 6, 7, 8, 9]

```
In [1]: 1 class Solution:
2       def merge(self,left,right):
3           i,j=0,0
4           result=[]
5           while(i<len(left) and j<len(right)):
6               if left[i]<=right[j]:
7                   result+=left[i]
8                   i+=1
9               else:
10                  result+=right[j]
11                  j+=1
12              result+=left[i:]+right[j:]
13              return result
14
15      def mergesort(self,arr):
16          if len(arr)<=1:
17              return arr
18          mid=len(arr)//2
19          left=self.mergesort(arr[:mid])
20          right=self.mergesort(arr[mid:])
21          return self.merge(left,right)
22 obj=Solution()
23 arr=[1,2,5,-1,-4,0,8,5]
24 obj.mergesort(arr)
```

Out[1]: [-4, -1, 0, 1, 2, 5, 5, 8]

```
In [1]: 1 class Solution:
2         def quicksort(self,arr,left,right):
3             if left<right:
4                 pos=self.partition(arr,left,right)
5                 self.quicksort(arr,left,pos-1)
6                 self.quicksort(arr,pos+1,right)
7             return arr
8         def partition(self,arr,left,right):
9             i=left
10            j=right-1
11            pivot=arr[right]
12            while i<j:
13                while i<right and arr[i]<pivot:
14                    i+=1
15                while arr[j]>=pivot and j>=i:
16                    j-=1
17                if i<j:
18                    arr[i],arr[j]=arr[j],arr[i]
19            if arr[i]>pivot:
20                arr[i],arr[right]=arr[right],arr[i]
21            return i
22 obj=Solution()
23 arr=[22,11,88,66,55,77,33,44,44,44,4]
24 obj.quicksort(arr,0,len(arr)-1)
```

Out[1]: [4, 11, 22, 33, 44, 44, 44, 55, 66, 77, 88]

```
In [5]: 1 class Solution:
2         def quick(self,arr):
3             if len(arr)<=1:
4                 return arr
5             else:
6                 pivot=arr[0]
7                 left=[i for i in arr[1:] if i<=pivot]
8                 right=[i for i in arr[1:] if i>pivot]
9                 return self.quick(left)+[pivot]+self.quick(right)
10 obj=Solution()
11 obj.quick([1,2,5,-1,-4,0,8,5])
```

Out[5]: [-4, -1, 0, 1, 2, 5, 5, 8]

```
In [11]: 1 class Solution:
2         def quicksort(self,arr,left,right):
3             if left<right:
4                 pos=self.partition(arr,left,right)
5                 self.quicksort(arr,left,pos-1)
6                 self.quicksort(arr,pos+1,right)
7             return arr
8         def partition(self,arr,left,right):
9             i=left
10            j=right-1
11            pivot=arr[right]
12            while i<j:
13                while i<right and arr[i]<pivot:
14                    i+=1
15                while j>=i and arr[j]>pivot:
16                    j-=1
17                if i<j:
18                    arr[i],arr[j]=arr[j],arr[i]
19            if arr[i]>pivot:
20                arr[i],arr[right]=arr[right],arr[i]
21            return i
22 obj=Solution()
23 arr=[1,2,5,-1,-4,0,8,5]
24 obj.quicksort(arr,0,len(arr)-1)
```

Out[11]: [-4, -1, 0, 1, 2, 5, 5, 8]

```
In [13]: 1 class Solution:
2         def divide(self,arr):
3             if len(arr)<=1:
4                 return arr
5             mid=len(arr)//2
6             left=self.divide(arr[:mid])
7             right=self.divide(arr[mid:])
8             return self.conquer(left,right)
9
10        def conquer(self,left,right):
11            res=[]
12            i,j=0,0
13            while(i<len(left) and j<len(right)):
14                if left[i]<right[j]:
15                    res+=left[i]
16                    i+=1
17                else:
18                    res+=right[j]
19                    j+=1
20            res+=left[i:]+right[j:]
21            return res
22 obj=Solution()
23 obj.divide([1,2,5,-1,-4,0,8,5])
```

Out[13]: [-4, -1, 0, 1, 2, 5, 5, 8]

```
In [1]: 1 from collections import Counter
2 def TreeConstructor(strArr):
3     parent = []
4     child = []
5
6     for i in strArr:
7         child.append(int(i[1]))
8         parent.append(int(i[3]))
9     for k,v in Counter(parent).items():
10         if v > 2:
11             return False
12     for k,v in Counter(child).items():
13         if v > 1:
14             return False
15     return True
16
17 print(TreeConstructor(["(1,2)", "(2,4)", "(5,7)", "(7,2)", "(9,5)"]))
18 print(TreeConstructor(["(1,2)", "(3,2)", "(2,12)", "(5,2)"]))
```

True
False

```
In [2]: 1 def Calculator(strParam):
2         return int(eval(strParam))
3 print(Calculator("6*(4/2)+3*1"))
```

15

```
In [8]: 1 class Sorting:
2         def bubbleSort(self,arr):
3             print(["unsorted array : ",arr])
4             l=len(arr)
5             for i in range(1):
6                 flag=0
7                 for j in range(l-i-1):
8                     if arr[j+1]<arr[j]:
9                         arr[j],arr[j+1]=arr[j+1],arr[j]
10                    flag=1
11             if flag==0:
12                 return ["sorted array : ",arr]
13             return ["sorted array : ",arr]
14 arr=[-5,2,5,1,7,9,0,-1,-1,0]
15 obj=Sorting()
16 obj.bubbleSort(arr)
```

['unsorted array : ', [-5, 2, 5, 1, 7, 9, 0, -1, -1, 0]]

Out[8]: ['sorted array : ', [-5, -1, -1, 0, 0, 1, 2, 5, 7, 9]]

```
In [11]: 1 class Sorting:
2         def insertionSort(self,arr):
3             print(["unsorted array : ",arr])
4             l=len(arr)
5             for i in range(1,l):
6                 gap=i
7                 value=arr[i]
8                 while gap>0 and arr[gap-1]>value:
9                     arr[gap]=arr[gap-1]
10                    gap-=1
11                arr[gap]=value
12            return ["sorted array : ",arr]
13 arr=[-5,2,5,1,7,9,0,-1,-1,0]
14 obj=Sorting()
15 obj.insertionSort(arr)
```

['unsorted array : ', [-5, 2, 5, 1, 7, 9, 0, -1, -1, 0]]

Out[11]: ['sorted array : ', [-5, -1, -1, 0, 0, 1, 2, 5, 7, 9]]

```
In [17]: 1 class Sorting:
2         def selection(self,arr):
3             print(["unsorted array : ",arr])
4             l=len(arr)
5             for i in range(l-1):
6                 min_index=i
7                 for j in range(i+1,l):
8                     if arr[j]<arr[min_index]:
9                         min_index=j
10                arr[min_index],arr[i]=arr[i],arr[min_index]
11            return ["sorted array : ",arr]
12 arr=[-5,2,5,1,7,9,0,-1,-1,0]
13 obj=Sorting()
14 obj.selection(arr)
```

['unsorted array : ', [-5, 2, 5, 1, 7, 9, 0, -1, -1, 0]]

Out[17]: ['sorted array : ', [-5, -1, -1, 0, 0, 1, 2, 5, 7, 9]]

```
In [31]: 1 class sorting:
2         print(["unsorted array : ",arr])
3         def divide(self,arr):
4             if len(arr)<=1:
5                 return arr
6             mid=len(arr)//2
7             left=self.divide(arr[:mid])
8             right=self.divide(arr[mid:])
9             return self.conquer(left,right)
10        def conquer(self,left,right):
11            i,j=0,0
12            res=[]
13            while i<len(left) and j<len(right):
14                if left[i]<right[j]:
15                    res+=left[i]
16                    i+=1
17                else:
18                    res+=right[j]
19                    j+=1
20            res+=left[i:]+right[j:]
21            return res
22        arr=[-5,2,5,1,7,9,0,-1,-1,0]
23        obj=sorting()
24        obj.divide(arr)

['unsorted array : ', [-5, 2, 5, 1, 7, 9, 0, -1, -1, 0]]
```

```
Out[31]: [-5, -1, -1, 0, 0, 1, 2, 5, 7, 9]
```

```
In [39]: 1 class Sorting:
2         def quick(self,arr):
3             self.a=5
4             if len(arr)<=1:
5                 return arr
6             pivot=arr[0]
7             left=[i for i in arr[1:] if i<pivot]
8             right=[i for i in arr[1:] if i>=pivot]
9             return self.quick(left)+[pivot]+self.quick(right)
10        obj=Sorting()
11        arr=[-5,2,5,1,7,9,0,-1,-1,0]
12        obj.quick(arr)
```

```
Out[39]: [-5, -1, -1, 0, 0, 1, 2, 5, 7, 9]
```

```
In [1]: 1 class Sorting:
2         def quick(self,arr,left,right):
3             if left<right:
4                 pos=self.partition(arr,left,right)
5                 self.quick(arr,left,pos-1)
6                 self.quick(arr,pos+1,right)
7             return arr
8         def partition(self,arr,left,right):
9             i=left
10            j=right-1
11            pivot=arr[right]
12            while i<j:
13                while i<right and arr[i]<pivot:
14                    i+=1
15                while j>=i and arr[j]>=pivot:
16                    j-=1
17                if i<=j:
18                    arr[i],arr[j]=arr[j],arr[i]
19            if arr[i]>pivot:
20                arr[i],arr[right]=arr[right],arr[i]
21            return i
22        obj=Sorting()
23        arr=[-5,2,5,1,7,9,0,-1,-1,0]
24        obj.quick(arr,0,len(arr)-1)
```

```
Out[1]: [-5, -1, -1, 0, 0, 1, 2, 5, 7, 9]
```

```
In [2]: 1 class Sorting:
2         def buildHeap(self,arr,n):
3             for i in range(n//2-1,-1,-1):
4                 self.heapify(arr,n,i)
5             print("maxheap:",arr)
6         def heapify(self,arr,n,i):
7             left=2*i+1
8             right=2*i+2
9             largest=i
10            if left<n and arr[largest]<arr[left]:
11                largest=left
12            if right<n and arr[largest]<arr[right]:
13                largest=right
14            if largest!=i:
15                arr[largest],arr[i]=arr[i],arr[largest]
16                self.heapify(arr,n,largest)
17        def heapSort(self,arr,n):
18            self.buildHeap(arr,n)
19            for i in range(n-1,-1,-1):
20                arr[i],arr[0]=arr[0],arr[i]
21                self.heapify(arr,i,0)
22            return "sorted array is : ",arr
23 arr=[4,2,7,45,-1,-34,0,0,23]
24 n=len(arr)
25 obj=Sorting()
26 obj.heapSort(arr,n)
```

maxheap: [45, 23, 7, 4, -1, -34, 0, 0, 2]

Out[2]: ('sorted array is : ', [-34, -1, 0, 0, 2, 4, 7, 23, 45])

```
In [1]: 1 class Sorting:
2         def bubble(self,arr):
3             l=len(arr)
4             for i in range(l):
5                 flag=1
6                 for j in range(l-i-1):
7                     if arr[j]>arr[j+1]:
8                         arr[j],arr[j+1]=arr[j+1],arr[j]
9                     flag=0
10                if flag==1:
11                    return arr
12 arr=[3,-1,2,-2,2,0,9,18,10,-45]
13 obj=Sorting()
14 obj.bubble(arr)
```

Out[1]: [-45, -2, -1, 0, 2, 2, 3, 9, 10, 18]

```
In [8]: 1 class Sorting:
2         def insertion(self,arr):
3             l=len(arr)
4             for i in range(1,l):
5                 value=arr[i]
6                 gap=i
7                 while gap>0 and arr[gap-1]>value:
8                     arr[gap]=arr[gap-1]
9                     gap-=1
10                arr[gap]=value
11            return arr
12 arr=[3,-1,2,-2,2,0,9,18,10,-45]
13 obj=Sorting()
14 obj.insertion(arr)
```

Out[8]: [-45, -2, -1, 0, 2, 2, 3, 9, 10, 18]

```
In [13]: 1 class Sorting:
2         def selection(self,arr):
3             l=len(arr)
4             for i in range(l-1):
5                 ind=i
6                 for j in range(i+1,l):
7                     if arr[j]<arr[ind]:
8                         ind=j
9                 arr[ind],arr[i]=arr[i],arr[ind]
10            return arr
11 arr=[3,-1,2,-2,2,0,9,18,10,-45]
12 obj=Sorting()
13 obj.selection(arr)
```

Out[13]: [-45, -2, -1, 0, 2, 2, 3, 9, 10, 18]

```
In [15]: 1 class Sorting:
2         def quick(self,arr):
3             if len(arr)<=1:
4                 return arr
5             pivot=arr[0]
6             left=[i for i in arr[1:] if i<pivot]
7             right=[i for i in arr[1:] if i>=pivot]
8             return self.quick(left)+[pivot]+self.quick(right)
9 arr=[3,-1,2,-2,2,0,9,18,10,-45]
10 obj=Sorting()
11 obj.quick(arr)
```

Out[15]: [-45, -2, -1, 0, 2, 2, 3, 9, 10, 18]

```
In [17]: 1 class Sorting:
2         def divide(self,arr):
3             if len(arr)<=1:
4                 return arr
5             mid=len(arr)//2
6             left=self.divide(arr[:mid])
7             right=self.divide(arr[mid:])
8             return self.merge(left,right)
9         def merge(self,left,right):
10            i,j=0,0
11            res=[]
12            while i<len(left) and j<len(right):
13                if left[i]<right[j]:
14                    res+=left[i]
15                    i+=1
16                else:
17                    res+=right[j]
18                    j+=1
19            res+=left[i:]+right[j:]
20            return res
21 arr=[3,-1,2,-2,2,0,9,18,10,-45]
22 obj=Sorting()
23 obj.divide(arr)
```

Out[17]: [-45, -2, -1, 0, 2, 2, 3, 9, 10, 18]

```
In [1]: 1 class Sorting:
2         def quick(self,arr,left,right):
3             if left<right:
4                 pos=self.partition(arr,left,right)
5                 self.quick(arr,left,pos-1)
6                 self.quick(arr,pos+1,right)
7             return arr
8         def partition(self,arr,left,right):
9             i=left
10            j=right-1
11            pivot=arr[right]
12            while i<j:
13                while i<right and arr[i]<pivot:
14                    i+=1
15                while j>=i and arr[j]>pivot:
16                    j-=1
17                if i<j:
18                    arr[i],arr[j]=arr[j],arr[i]
19            if arr[i]>pivot:
20                arr[i],arr[right]=arr[right],arr[i]
21            return i
22 arr=[3,-1,2,-2,2,0,9,18,10,-45]
23 obj=Sorting()
24 obj.quick(arr,0,len(arr)-1)
```

Out[1]: [-45, -2, -1, 0, 2, 2, 3, 9, 10, 18]

```
In [5]: 1 class Sorting:
2         def buildheap(self,arr,n):
3             for i in range(n//2-1,-1,-1):
4                 self.heapify(arr,n,i)
5         def heapify(self,arr,n,i):
6             largest=i
7             left=2*i+1
8             right=2*i+2
9             if left<n and arr[left]>arr[largest]:
10                largest=left
11            if right<n and arr[right]>arr[largest]:
12                largest=right
13            if largest!=i:
14                arr[largest],arr[i]=arr[i],arr[largest]
15                self.heapify(arr,n,largest)
16         def heapsort(self,arr,n):
17             self.buildheap(arr,n)
18             for i in range(n-1,-1,-1):
19                 arr[0],arr[i]=arr[i],arr[0]
20                 self.heapify(arr,i,0)
21             return arr
22 arr=[3,-1,2,-2,2,0,9,18,10,-45]
23 obj=Sorting()
24 obj.heapsort(arr,len(arr)-1)
```

Out[5]: [-2, -1, 0, 2, 2, 3, 9, 10, 18, -45]

```
In [4]: 1 class Sorting:
2         def divide(self, arr):
3             if len(arr) <= 1:
4                 return arr
5             mid = len(arr) // 2
6             left = self.divide(arr[:mid])
7             right = self.divide(arr[mid:])
8             return self.merge(left, right)
9         def merge(self, left, right):
10            i, j = 0, 0
11            res = []
12            while i < len(left) and j < len(right):
13                if left[i] < right[j]:
14                    res += [left[i]]
15                    i += 1
16                else:
17                    res += [right[j]]
18                    j += 1
19            res += left[i:] + right[j:]
20            return res
21 obj = Sorting()
22 arr = [9, 3, 1, 8, 4, 6, 0, -1, -4, -2, -1, -1]
23 obj.divide(arr)
```

Out[4]: [-4, -2, -1, -1, -1, 0, 1, 3, 4, 6, 8, 9]

```
In [1]: 1 class Sorting:
2         def quick(self, arr):
3             if len(arr) <= 1:
4                 return arr
5             pivot = arr[0]
6             left = [i for i in arr[1:] if i < pivot]
7             right = [i for i in arr[1:] if i >= pivot]
8             return self.quick(left) + [pivot] + self.quick(right)
9 obj = Sorting()
10 arr = [9, 3, 1, 8, 4, 6, 0, -1, -4, -2, -1, -1]
11 obj.quick(arr)
```

Out[1]: [-4, -2, -1, -1, -1, 0, 1, 3, 4, 6, 8, 9]

```
In [9]: 1 class Sorting:
2         def quick(self, arr, left, right):
3             if left < right:
4                 pos = self.partition(arr, left, right)
5                 self.quick(arr, left, pos - 1)
6                 self.quick(arr, pos + 1, right)
7             return arr
8         def partition(self, arr, left, right):
9             i = left
10            j = right - 1
11            pivot = arr[right]
12            while i < j:
13                while i < right and arr[i] < pivot:
14                    i += 1
15                while j >= i and arr[j] >= pivot:
16                    j -= 1
17                if i < j:
18                    arr[i], arr[j] = arr[j], arr[i]
19            if arr[i] > pivot:
20                arr[i], arr[right] = arr[right], arr[i]
21            return i
22 obj = Sorting()
23 arr = [9, 3, 1, 8, 4, 6, 0, -1, -4, -2, -1, -1]
24 obj.quick(arr, 0, len(arr) - 1)
```

Out[9]: [-4, -2, -1, -1, -1, 0, 1, 3, 4, 6, 8, 9]

```
In [10]: 1 class Sorting:
2         def bubble(self, arr):
3             l = len(arr)
4             for i in range(1):
5                 flag = 1
6                 for j in range(l - i - 1):
7                     if arr[j] > arr[j + 1]:
8                         arr[j], arr[j + 1] = arr[j + 1], arr[j]
9                     flag = 0
10                if flag == 1:
11                    return arr
12            return arr
13 obj = Sorting()
14 arr = [9, 3, 1, 8, 4, 6, 0, -1, -4, -2, -1, -1]
15 obj.bubble(arr)
```

Out[10]: [-4, -2, -1, -1, -1, 0, 1, 3, 4, 6, 8, 9]

```
In [11]: 1 class Sorting:
2         def insertion(self,arr):
3             l=len(arr)
4             for i in range(1,l):
5                 gap=i
6                 val=arr[gap]
7                 while gap>0 and arr[gap-1]>val:
8                     arr[gap]=arr[gap-1]
9                     gap-=1
10                arr[gap]=val
11            return arr
12 obj=Sorting()
13 arr=[9,3,1,8,4,6,0,-1,-4,-2,-1,-1]
14 obj.insertion(arr)
```

Out[11]: [-4, -2, -1, -1, -1, 0, 1, 3, 4, 6, 8, 9]

```
In [18]: 1 class Sorting:
2         def selection(self,arr):
3             l=len(arr)
4             for i in range(1):
5                 index=i
6                 for j in range(i+1,l):
7                     if arr[j]<arr[index]:
8                         index=j
9                 arr[i],arr[index]=arr[index],arr[i]
10            return arr
11 obj=Sorting()
12 arr=[9,3,1,8,4,6,0,-1,-4,-2,-1,-1]
13 obj.selection(arr)
```

Out[18]: [-4, -2, -1, -1, -1, 0, 1, 3, 4, 6, 8, 9]

```
In [2]: 1 class Solution:
2         def quick(self,arr):
3             if len(arr)<=1:
4                 return arr
5             pivot=arr[0]
6             left=[i for i in arr[1:] if i<pivot]
7             right=[i for i in arr[1:] if i>=pivot]
8             return self.quick(left)+[pivot]+self.quick(right)
9 obj=Solution()
10 arr=[5,43,7,1,77,0,-8,34,-1213123]
11 obj.quick(arr)
```

Out[2]: [-1213123, -8, 0, 1, 5, 7, 34, 43, 77]

```
In [3]: 1 class Solution:
2         def quicksort(self,arr,left,right):
3             if left<right:
4                 pos=self.partition(arr,left,right)
5                 self.quicksort(arr,left,pos-1)
6                 self.quicksort(arr,pos+1,right)
7             return arr
8         def partition(self,arr,left,right):
9             pivot=arr[right]
10            i=left
11            j=right-1
12            while i<j:
13                while i<right and arr[i]<pivot:
14                    i+=1
15                while j>i and arr[j]>=pivot:
16                    j-=1
17                if i<j:
18                    arr[i],arr[j]=arr[j],arr[i]
19            if arr[i]>pivot:
20                arr[i],arr[right]=arr[right],arr[i]
21            return i
22 obj=Solution()
23 arr=[2,4,64,12,231,-89,0,654]
24 obj.quicksort(arr,0,len(arr)-1)
```

Out[3]: [-89, 0, 2, 4, 12, 64, 231, 654]


```
In [4]: 1 class Solution:
2     def divide(self, arr):
3         if len(arr) <= 1:
4             return arr
5         mid = len(arr) // 2
6         left = self.divide(arr[:mid])
7         right = self.divide(arr[mid:])
8         return self.conquer(left, right)
9     def conquer(self, left, right):
10        i, j = 0, 0
11        res = []
12        while i < len(left) and j < len(right):
13            if left[i] < right[j]:
14                res += [left[i]]
15                i += 1
16            else:
17                res += [right[j]]
18                j += 1
19        res += left[i:] + right[j:]
20        return res
21 obj = Solution()
22 arr = [1, 23, 5345121, 12, 213, -98, 0, 456, 34]
23 obj.divide(arr)
```

Out[4]: [-98, 0, 1, 12, 23, 34, 213, 456, 5345121]

```
In [20]: 1 class Solution:
2     def buildheap(self, arr, n):
3         for i in range(n // 2 - 1, -1, -1):
4             self.heapify(arr, n, i)
5     def heapify(self, arr, n, i):
6         largest = i
7         left = 2 * i + 1
8         right = 2 * i + 2
9         if left < n and arr[left] > arr[largest]:
10            largest = left
11        if right < n and arr[right] > arr[largest]:
12            largest = right
13        if largest != i:
14            arr[largest], arr[i] = arr[i], arr[largest]
15            self.heapify(arr, n, largest)
16    def heapsort(self, arr, n):
17        self.buildheap(arr, n)
18        for i in range(n - 1, -1, -1):
19            arr[0], arr[i] = arr[i], arr[0]
20            self.heapify(arr, i, 0)
21        return arr
22 obj = Solution()
23 arr = [1, 23, 5345121, 12, 213, -98, 0, 456, 34, 45, 4567, 123125, 1]
24 obj.heapsort(arr, len(arr))
```

Out[20]: [-98, 0, 1, 1, 12, 23, 34, 45, 213, 456, 4567, 123125, 5345121]

```
In [3]: 1 class Sorting:
2     def buildheap(self, arr, n):
3         for i in range(n // 2 - 1, -1, -1):
4             self.heapify(arr, n, i)
5     def heapify(self, arr, n, i):
6         largest = i
7         left = 2 * i + 1
8         right = 2 * i + 2
9         if left < n and arr[left] > arr[largest]:
10            largest = left
11        if right < n and arr[right] > arr[largest]:
12            largest = right
13        if largest != i:
14            arr[largest], arr[i] = arr[i], arr[largest]
15            self.heapify(arr, n, largest)
16    def heapsort(self, arr, n):
17        self.buildheap(arr, n)
18        for i in range(n - 1, -1, -1):
19            arr[0], arr[i] = arr[i], arr[0]
20            self.heapify(arr, i, 0)
21        return arr
22 arr = [1, 23, 534, 12, 213, -98, 0, 456, 34]
23 obj = Sorting()
24 obj.heapsort(arr, len(arr))
```

Out[3]: [-98, 0, 1, 12, 23, 34, 213, 456, 534]

```
In [2]: 1 class Solution:
2         def buildTree(self, arr, n):
3             for i in range(n//2-1, -1, -1):
4                 self.heapify(arr, n, i)
5             print(arr)
6         def heapify(self, arr, n, i):
7             largest=i
8             left=2*i+1
9             right=2*i+2
10            if left<n and arr[left]>arr[largest]:
11                largest=left
12            if right<n and arr[right]>arr[largest]:
13                largest=right
14            if largest!=i:
15                arr[largest], arr[i]=arr[i], arr[largest]
16                self.heapify(arr, n, largest)
17        def heapsort(self, arr, n):
18            self.buildTree(arr, n)
19            for i in range(n-1, -1, -1):
20                arr[i], arr[0]=arr[0], arr[i]
21                self.heapify(arr, i, 0)
22            return arr
23 obj=Solution()
24 arr=[1,23,534,12,213,-98,0,456,34]
25 obj.heapsort(arr, len(arr))
```

[534, 456, 1, 34, 213, -98, 0, 12, 23]

Out[2]: [-98, 0, 1, 12, 23, 34, 213, 456, 534]

```
In [9]: 1 class Solution:
2         def divide(self, arr):
3             if len(arr)<=1:
4                 return arr
5             mid=len(arr)//2
6             left=self.divide(arr[:mid])
7             right=self.divide(arr[mid:])
8             return self.conquer(left, right)
9         def conquer(self, left, right):
10            i,j=0,0
11            res=[]
12            while i<len(left) and j<len(right):
13                if left[i]<right[j]:
14                    res+=left[i]
15                    i+=1
16                else:
17                    res+=right[j]
18                    j+=1
19            res+=left[i:]+right[j:]
20            return res
21 obj=Solution()
22 arr=[1,23,534,12,213,-98,0,456,34]
23 obj.divide(arr)
```

Out[9]: [-98, 0, 1, 12, 23, 34, 213, 456, 534]

```
In [10]: 1 class Solution:
2         def quick(self, arr):
3             if len(arr)<=1:
4                 return arr
5             pivot=arr[0]
6             left=[i for i in arr[1:] if i<pivot]
7             right=[i for i in arr[1:] if i>=pivot]
8             return self.quick(left)+[pivot]+self.quick(right)
9         obj=Solution()
10        arr=[1,23,534,12,213,-98,0,456,34]
11        obj.quick(arr)
```

Out[10]: [-98, 0, 1, 12, 23, 34, 213, 456, 534]

```
In [12]: 1 class Solution:
2         def quick(self, arr, left, right):
3             if left < right:
4                 pos = self.partition(arr, left, right)
5                 self.quick(arr, left, pos-1)
6                 self.quick(arr, pos+1, right)
7             return arr
8         def partition(self, arr, left, right):
9             i = left
10            j = right-1
11            pivot = arr[right]
12            while i < j:
13                while i < j and arr[i] < pivot:
14                    i += 1
15                while j >= i and arr[j] > pivot:
16                    j -= 1
17                if i < j:
18                    arr[i], arr[j] = arr[j], arr[i]
19            if arr[i] > pivot:
20                arr[i], arr[right] = arr[right], arr[i]
21            return i
22 obj = Solution()
23 arr = [1, 23, 534, 12, 213, -98, 0, 456, 34]
24 obj.quick(arr, 0, len(arr)-1)
```

```
Out[12]: [-98, 0, 1, 12, 23, 34, 213, 456, 534]
```

```
In [33]: 1 class ListNode:
2         def __init__(self, val):
3             self.val = val
4             self.next = None
5         class LinkedList:
6             def __init__(self):
7                 self.head = None
8             def printList(self):
9                 itr = self.head
10                if not itr: return 'empty sll'
11                res = ''
12                while itr:
13                    res += str(itr.val) + '-->' if itr.next else str(itr.val)
14                    itr = itr.next
15                return res
16            def sort(self):
17                itr = self.head
18                while itr:
19                    org = itr
20                    temp = itr.next
21                    while temp:
22                        if temp.val < org.val:
23                            org = temp
24                        temp = temp.next
25                    x = itr.val
26                    itr.val = org.val
27                    org.val = x
28                    itr = itr.next
29 obj = LinkedList()
```

```
In [34]: 1 arr = [4, 2, -1, 0, 3, 8, 5, 6]
2         obj.head = ListNode(arr[0])
3         temp = obj.head
4         for i in range(1, len(arr)):
5             temp.next = ListNode(arr[i])
6             temp = temp.next
```

```
In [35]: 1 obj.printList()
```

```
Out[35]: '4-->2-->-1-->0-->3-->8-->5-->6'
```

```
In [36]: 1 obj.head.next.next.val
```

```
Out[36]: -1
```

```
In [37]: 1 obj.sort()
```

```
In [38]: 1 obj.printList()
```

```
Out[38]: '-1-->0-->2-->3-->4-->5-->6-->8'
```

```
In [ ]: 1
```

```
In [ ]: 1
```

```
In [ ]: 1
```

```
In [ ]: 1
```

